iPRA with evaluation at witness discharge

Three inference families on raw terms

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a three-family presentation of full first-order intuitionistic primitive recursive arithmetic on raw scalar-iterator terms. The primitive families are clause use, paired introduction, and witness discharge with one elementary evaluation; universal reflexivity is the one explicit fixed axiom clause. There is no numerical normalization. The evaluation map contains the two existing iterator-evaluation patterns and identity as its fallback. We prove exact equivalence, on every open end sequent, with the reference calculus having separate evaluation and witness elimination. The inherited whole-domain interpretations with iPRA therefore retain actual equality and identity numerical composites. Both complete formula round trips and equiconsistency are included.

The complete raw checker has 1,892 normalized bytes, 643 significant tokens and 695 AST nodes, versus 1,995, 665 and 717 for its matched control. A zero-normal companion has 1,858 bytes, 633 tokens and 697 AST nodes. Both certificate predicates are primitive recursive without fuel. The improvement is a local rule fusion and a measured reduction in the stated checker family, not an absolute minimum or a claim to surpass the repository's different compact codecs. The archive includes complete checkers, proof translations, independent replay and the mathematical foundations needed for the bi-interpretation.

Contents

1 The result and its comparison target 3

2 The complete raw presentation 3

3 Exact equivalence on raw open sequents 5

4 Where equality, computation and induction come from 6

5 The smaller zero-normal companion 7

6 Full first-order iPRA and both interpretation composites 8

7 Complete certificate checkers 10

8 Measured comparison and executed evidence 12

9 Limitations and further research questions 13

10 Reproduction and attribution 14

A Complete raw acceptance checker 15

B Complete zero-normal acceptance checker 16

C Complementary introductions with one local assumption 17

D Arithmetic strength and both ordinary interpretation composites 22

E Internal scalar predecessor bootstrap 24

F From literal-equation induction to full intuitionistic PRA 28

G Pairing, primitive recursion, and internal numerical composites 34

1 The result and its comparison target

The proposed simplification is a calculus with **three inference families on completely raw terms**: use a retained clause, introduce a paired formula, and discharge a witness while performing one elementary evaluation. There is one explicit initial axiom clause, universal reflexivity. Numerical computation is never an implicit conversion rule. Neither full iterator normalization nor zero normalization is needed in this primary presentation.

The construction is compared with ProveIt's `ConciseIPRA` at commit `11814771b749`. The inspected frontier includes the seventy-three filed packages and all six archives then present in `docs/incoming`. In particular, the comparison includes `witness_discharge`, `clause_discharge`, `deterministic_opening`, `first_instance`, `inferred_witness`, `prefix_clauses`, and `substitutive_discharge`. The last group matters: a proposal that only removes the supplied equality proof or replacement term from the earlier Name rule would already be present there.

What is new here is the fusion of the existing *computation-and-equality evaluation rule* with witness elimination. In contrast, the earlier three-family Name presentations put both iterator equations into numerical normal forms. The incoming raw contextual-discharge variant retains a separate computation family. The present raw calculus has three families while keeping ordinary, nonnormalizing substitution.

An optional companion uses zero-normal terms and the existing single SeedEval pattern. It also has three families, and has the smaller complete checker. The raw variant is the cleaner recommendation when the priority is the trusted mathematical specification; the companion is preferable when normalized checker size is decisive. Both preserve the entire intuitionistic first-order theory, not merely its equations.

The result is a local simplification of the current presentation. It does not prove an absolute minimum over all axiomatizations, encodings or programming languages. Nor does it beat the repository's separate 508-byte compact codec, whose proof representation and validation boundary are different. The whole-program comparison in Section 8 makes these boundaries explicit.

2 The complete raw presentation

2.1 Syntax, clauses, and the one fixed axiom

There is one numerical sort. Raw terms and native formulas are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid D_{x,y}A.$$

Only u in h is bound by the iterator. The two variables of D are bound in its matrix. Bound variables are renamed before capture-avoiding substitution; the executable representation instead uses bound indices. All finite expressions of this grammar are terms or formulas: there is no normal-form side condition and no identification by computation. In particular, $I(0, a; u.h)$ and a are distinct terms.

A reusable clause has the form $[\vec{A}; y.B(y)]$, where $\vec{A}$ is empty or consists of one formula, and y is absent from that antecedent. Its ordinary reading is, respectively, $\forall y\, B(y)$ or $A \to \forall y\, B(y)$. A context is a finite list of clauses. The abbreviation Γ, A means addition of the unconditional constant clause $[\,; v.A]$, with v unused.

Every arithmetic proof begins with the explicitly counted clause

$$\boxed{\mathcal{R} = [\,; v.v = v].}$$

It supplies equality reflexivity by clause use. The native contradiction is the formula $0 = S0$. There are no other primitive axiom clauses.

2.2 One deterministic elementary evaluation

For arbitrary formulas E, F, define $\rho_E(F)$ by the following three disjoint cases. Every match is literal syntax, including equality of the displayed iterator bodies after the chosen representation of binders.

$$\rho_{s=t}\big(I(0, s; u.h) = r\big) = (r = t), \tag{Z}$$

$$\rho_{I(c,a;u.h)=t}\big(I(Sc, a; u.h) = r\big) = \big(r = h[t/u]\big), \tag{S}$$

$$\rho_E(F) = F \qquad \text{in every other case.} \tag{Id}$$

The zero case has a literal zero counter; the successor case has a successor constructor, so they cannot overlap. In the second case the entire predecessor iterator on the left of E must match the one obtained from the left of F by deleting its counter's outer successor. The replacement term, body, and counter are inferred from the two formulas; no term template or computation description is separate rule data.

The substitution $h[t/u]$ is a single ordinary syntactic substitution. The result is not normalized. An auxiliary formula E with a D root always selects Id. The identity case does not derive F without a premise: the rule below explicitly requires a proof of F.

2.3 All three inference families

Use. For any retained clause $[\vec{A}; y.B] \in \Gamma$ and term t,

$$\frac{\{\Gamma \vdash A : A \in \vec{A}\}}{\Gamma \vdash B[t/y]}. \tag{Use}$$

There is no proof premise for an unconditional clause, and one for a conditional clause. This is the only leaf mechanism; reflexivity is an instance of Use at $\mathcal{R}$.

Introduce.

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is literally an equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Intro}$$

Here z is fresh for Γ, the witness t, and the passive parameters of B. The witness test is on raw syntax. The progress case is restricted to literal equations; it does not introduce induction on quantified formulas.

Evaluate and discharge. For a fresh numerical parameter b, write $\mathcal{B}_b = [B(b,0); y.B(b,y)]$. The third family is

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{B}_b \vdash E \qquad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash \rho_E(F)}. \tag{Discharge}$$

The parameter b is fresh for Γ and the passive parameters of the major formula. Both branch proofs use the same extended context. The computed conclusion must be well scoped in the outer numerical scope; equivalently, it has no free occurrence of b. The intermediate formulas E, F may contain b.

These are the entire primitive rules of the calculus, denoted $\mathcal{E}_3$. There is no separate hypothesis, equality transport, computation, elimination, case, induction, normalization, weakening or cut instruction. The two nonidentity patterns inside ρ remain explicitly counted in the specification. Calling this "three families" does not conceal their existence or establish a lower bound.

3 Exact equivalence on raw open sequents

Let $\mathcal{V}_4$ be the same syntax, clauses, Use and Intro, with ordinary paired elimination and the following two-case Eval family in place of Discharge:

$$\frac{\Gamma \vdash s = t \quad \Gamma \vdash I(0, s; u.h) = r}{\Gamma \vdash r = t}, \qquad \frac{\Gamma \vdash I(c, a; u.h) = t \quad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]}.$$

Its ordinary paired elimination is

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, \mathcal{B}_b \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(A).$$

This is the repository's raw uniform-clause calculus: Eval is the inherited two-case arithmetic interface, and $\mathcal{R}$ replaces a separate reflexivity rule. The pinned Lean files `NativeEval.lean`, `NativeClausal.lean`, and `NativeUniformClauses.lean` establish the corresponding reference equivalences [1, 2]. The new fusion theorem below is proved here, rather than attributed to those existing formalizations.

Lemma 3.1 (Structural operations)**.** *Weakening and injective renaming of free parameters are admissible in $\mathcal{E}_3$ and $\mathcal{V}_4$.*

Proof. Induct on derivations, freshening a bound eigenparameter before adding an outer parameter or clause. Injective renaming preserves and reflects the literal matches in ρ, preserves the zero/successor constructors, commutes with capture-avoiding substitution, and preserves the Intro mode. Weakening changes none of that data. No assertion that arbitrary numerical substitution preserves an instruction's selected case is needed. □

Lemma 3.2 (A compound theorem with no arithmetic premise)**.** *For every clause context, $\Theta = D_{x,y}(0 = 0)$ is derivable in both systems, even if that context does not contain $\mathcal{R}$.*

Proof. Introduce at witness 0. The temporary progress assumption and the successor target are both $0 = 0$. Use that newly added unconditional assumption. Thus the proof uses only Intro and Use. □

Theorem 3.3 (Identity on the complete consequence relation)**.** *For every well-scoped raw clause context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{E}_3} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{V}_4} A.$$

Both translations are effective transformations of finite proof trees and preserve the entire end sequent literally. The statement does not require $\mathcal{R}$ *to be present.*

Proof. From left to right, translate a Discharge node's three premises. If its ρ computation selects Z or S, apply the corresponding Eval inside the witness scope. If it selects Id, the translated branch proof of F already has the required conclusion. Apply ordinary paired elimination. Its eigenparameter condition is exactly the outer-scope condition of Discharge. Use and Intro translate without change.

Conversely, translate a reference elimination node by Discharge. Use as auxiliary E the same compound major $D_{x,y}B$, weakened into the fresh branch. Since E is compound, $\rho_E(A) = A$, so the original conclusion is unchanged.

Translate either Eval node by Discharge with the fixed major proof of Θ from Lemma 3.2. Weaken its original equation premises into that fresh scope, using them as E, F. The intended Z or S pattern matches and reproduces the Eval conclusion. All data of that conclusion are from the outer scope. Recursing over the finite tree, with fresh parameters at every inserted scope, proves the two transformations. □

Direct ordinary soundness. Read $D_{x,y}B$ as $\exists x(B(x,0) \to \forall y\, B(x,y))$ and read equality as actual equality. Eliminate the existential at b. In case Z, the zero computation equation and $s = t$ turn the second branch equation into $r = t$. In case S, successor computation and the predecessor equation give $r = h[t/u]$. Id simply retains the branch formula. Each conclusion can leave the witness scope because its free occurrence of b is forbidden. This is intuitionistic reasoning and does not use a truth-table argument or a restriction to the standard natural numbers.

4 Where equality, computation and induction come from

The exact comparison theorem already transfers completeness. The following derivations expose the arithmetic content and show that no equality law is being silently supplied by the checker. Write $J_s = I(0, s; u.0)$. All Eval applications below can be expanded by Theorem 3.3 into the three-family calculus.

Joining equations. From $s = t$ and reflexivity of J_s, Z yields $J_s = t$. From $s = r$ and that equation, a second Z yields $t = r$. Consequently $s = t, s = r \vdash t = r$. Reflexivity supplies symmetry, and joining with symmetry supplies transitivity.

Both computation equations. Z on reflexivity of a and of $I(0, a; u.h)$ yields $I(0, a; u.h) = a$. S on reflexivity of the predecessor and successor iterators yields

$$I(Sc, a; u.h) = h[I(c, a; u.h)/u].$$

These are theorems. They are not reductions performed by raw substitution.

Transport in any term. Suppose $s = t$ and $h(s) = r$. Put $J = I(0, s; u.h)$ and $K = I(S0, s; u.h)$. Zero computation gives $J = s$; successor computation and that equation give $K = h(s)$, using S. Joining gives $K = r$. The zero Eval on $s = t$ and reflexivity of J gives $J = t$. The successor Eval on $J = t$ and $K = r$ now yields $r = h(t)$. This is the inherited crossed transport rule in an arbitrary term template. It supplies congruence and transport in an equation. Transport through a compound formula follows by external induction on its D depth, opening the old D, introducing the new D at the fresh nonzero-syntax witness, and transporting its base and successor instances.

Equation induction. For a literal equation $E(y)$, choose an unused witness coordinate x. The progress instance of Intro at witness 0 turns $\Gamma, E(z) \vdash E(Sz)$ into $D_{x,y}E(y)$. Derived ordinary elimination opens the clause $[E(0); y.E(y)]$. A supplied base proof of $E(0)$ and Use at any term t derive $E(t)$. This is contextual induction on equations. No induction on compound formulas has been added.

The case mode omitted at a literal zero witness. It is recovered at the raw witness $w = I(0, 0; u.0)$, which is not literally 0 and is provably equal to 0. Transport converts the local assumption and successor target between the witnesses w and 0. Ordinary-assumption cut, proved by induction on derivations after freshening parameters, inserts the original case proof. Thus the single introduction recovers both reference introductions. This argument uses raw syntax, and is one reason not to silently identify computed terms in the primary calculus.

The full native logical interface and the arithmetic bridge are recorded in Sections 6 and the attributed appendices. The logical macros, predecessor construction, equality reflection and pairing compiler are inherited; the new theorem changes their proof interface without changing their conclusions.

5 The smaller zero-normal companion

There is a second presentation, $\mathcal{E}_3^0$, with the same three inference families and the same fixed axiom clause. Its terms are canonical only for the deletion

$$I(0, a; u.h) \longrightarrow a.$$

Let N_0 perform this deletion hereditarily. Every rule substitution is followed by N_0. A successor-headed iterator is not unfolded by this syntactic operation. The evaluation map is instead

$$\sigma_E(F) = \begin{cases} r = N_0(h[I(c, t; u.h)/u]), & E \equiv (s = t), \quad F \equiv (I(Sc, s; u.h) = r), \\ F, & \text{otherwise.} \end{cases}$$

The initializer in the equation E must match the initializer of the iterator in F. The previous iterate is constructed from the replacement initializer t; it is not an additional premise. This is precisely the existing SeedEval rule, executed only at witness discharge.

Use and Intro are those of Section 2, with normalized substitution and the literal witness test on canonical syntax. Discharge exports $\sigma_E(F)$ instead of $\rho_E(F)$, subject to the same outer scope condition. There is one arithmetic pattern and the identity case.

The proof of Theorem 3.3 applies verbatim with SeedEval in place of the two raw Eval cases. Its separate-rule control has Use, Intro, Open and SeedEval. The compound major still recovers Open, and Θ still recovers SeedEval. In the reverse simulation no arbitrary substitution into an existing derivation is assumed.

The arithmetic interface is unchanged. At counter 0, SeedEval reads

$$a = b,\ I(S0, a; u.h) = r \quad \Longrightarrow \quad r = N_0(h[b/u]).$$

Taking $h = u$ and using two such evaluations derives the joining rule for equations. Taking arbitrary h derives transport by the same two-step pivot construction. Two reflexivities give successor computation. Zero computation is canonical syntax. This interface is formalized in the pinned `NativeZeroSeed.lean` as `NVPrf`; the equivalence `nvprf_iff_pra` is already in that source [3]. The clause representation and its fixed reflexivity clause are formalized in `NativeZeroClausal.lean`. The present fusion is a new written equivalence on top of those interfaces, not a claim that its new checker has been verified by Lean.

Only structural normalization is used. Recursively normalize an iterator's children. If its normalized counter is 0, return its normalized initializer; otherwise rebuild the iterator. Every contraction deletes nodes, and none duplicates a body. Structural induction proves termination, idempotence and substitution coherence

$$N_0(N_0(t)[\tau]) = N_0(t[\tau]).$$

Thus this companion does not reinstate the full computation normalizer removed from the main presentation.

6 Full first-order iPRA and both interpretation composites

The reference theory is full first-order intuitionistic primitive recursive arithmetic: intuitionistic equality logic; a symbol for each fixed finite primitive-recursive description and its defining equations; separation of zero from successors; successor injectivity; and induction for every quantifier-free formula, with parameters and retained assumptions. In particular, the target is not merely equational PRA or a negative fragment of iPRA.

6.1 The intermediate ordinary scalar theory

Let T_I be ordinary intuitionistic first-order equality logic over the staged scalar-iterator signature. Its nonlogical axioms and rules are exactly the two iterator equations, numerical coverage, separation, and contextual equation induction:

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$
$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot,$$
$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \quad (E \text{ an equation},\ z \text{ fresh}).$$

The displayed separation sentence is essential to the ordinary realization: positive equations by themselves do not prohibit a one-element structure. It is not an extra hidden axiom of the native calculus, where $0 = S0$ is the designated contradiction and explosion is derived.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (D_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Unconditional clauses translate to universals and conditional clauses to implications into universals. All numerical domains are the whole domain, defined by $x = x$, and equality is actual equality.

6.2 Explicit logical macros

Put $C(v,a,b) = I(v,a;u.b)$, with u unused, and $p(z) = I(z,0;u.C(u,1,0))$. The derived computation equations give the selector endpoints and $p(1) = 1$, $p(2) = 0$. For a formula, define its gate recursively by

$$G_v(s = t) = (C(v,0,s) = C(v,0,t)), \qquad G_v(D_{x,y}B) = D_{x,y}G_vB,$$

with fresh bound variables. It is provable at $v = 0$ and equivalent to the original formula at every successor. Formulas can be padded by $A \dashv\vdash D_{u,y}G_yA$.

The formula selector $M_v(A,B)$ uses C on corresponding sides of two equations and distributes through aligned D binders. If one formula is an equation and the other has a D binder, pad the equation using the preceding gate before continuing. This finite recursive definition, spelled out in Appendix C, gives $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$. Write $Q_yA = D_{u,y}A$ with u unused. One complete macro inventory is

$$\begin{aligned}
\bot_* &= (0 = S0),\\
A \to_* B &= D_{u,y}M_y(A,B),\\
A \wedge_* B &= D_{u,y}G_yM_{p(y)}(A,B),\\
\exists_* x\, A(x) &= D_{x,y}G_yA(x),\\
A \vee_* B &= \exists_* x\, M_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* Q_yA(y).
\end{aligned}$$

Every newly bound variable is fresh, except the explicitly displayed argument of the formula being bound. These macros have all the usual intuitionistic introduction and elimination rules. For example, implication uses the selector's zero row as its assumption and its successor rows as its conclusion; conjunction is read at the two controls $1, 2$; existential quantification uses the true zero gate. Universal elimination uses $A(0)$ to activate the Q clause. Under $0 = S0$, selector computation makes any two terms equal, and induction on finite formula syntax yields explosion.

The appended inherited logical development proves these claims. Its equivalent universal macro uses a gated paired conjunct; the macro displayed here is the more direct $A(0) \wedge Q_yA(y)$ also used by the current Lean development. Both have the ordinary universal reading.

6.3 The numerical translations

From scalar terms to iPRA, translate each fixed body $h(u,\vec{p})$ by structural recursion on its finite description. Its iterator is the primitive-recursive operation

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

No number is asked to encode a function on which a universal total evaluator operates.

In the other direction, the inherited scalar development first derives predecessor from scalar iteration and equation induction. It then proves successor injectivity, constructs an equality indicator that reflects actual equality internally, and obtains induction on all quantifier-free formulas. Pairing and its projections are derived next. For a fixed primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n,f(n,\vec{p}),\vec{p}),$$

the scalar state stores a pair consisting of the counter and the current value. If L, H are its two

projections, set

$$W(v,\vec{p}) = \pi(SL(v), h^*(L(v), H(v), \vec{p})),$$
$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); v.W(v,\vec{p})),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$, and the two recursion equations for f^* follow. The order of this argument matters: the predecessor and equality-reflection proofs do not presuppose the general primitive-recursion compiler. The appended foundations supply that order and the necessary internal proofs [4, 5, 6, 7, 8].

Theorem 6.1 (Required bi-interpretability and equiconsistency)**.** *The ordinary realizations of $\mathcal{E}_3$ and $\mathcal{E}_3^0$ are bi-interpretable with full first-order iPRA. Both numerical interpretation domains are whole, equality is actual equality, and both numerical composite isomorphisms have graph $x = y$. Both formula composites are provably equivalent to the identity. Each presentation is equiconsistent with iPRA.*

Proof. Theorem 3.3 transfers the full native consequence relation to the raw uniform-clause reference, whose scalar and iPRA interpretations are the ones just described. The zero companion has the same exact comparison with its SeedEval reference.

For the numerical composites, induction on a fixed finite primitive- recursive description shows that its double translation satisfies the original defining equations. Internal equation induction proves uniqueness of that fixed recursion. The analogous induction on scalar syntax proves the other composite. These are statements for arbitrary internal counters, not just for standard numeral instances.

The crucial native formula composite is

$$D_{x,y}B \dashv\vdash \exists_* x(B(x,0) \to_* \forall_* y\, B(x,y)).$$

Forward, open its witness clause, assume its base, and use that clause at a fresh variable before introducing the ordinary macros. Backward, eliminate the existential at a fresh witness and use case introduction with the supplied implication and universal at the successor instance. The other formula composites follow from the selector and gate identities and induction on finite formulas. The zero companion adds the internally provable equivalence of a term with its zero-normal form.

The translations preserve contradiction: ordinary falsity maps to $0 = S0$, and the ordinary reading of $0 = S0$ implies falsity by separation. Effective translations of finite derivations in both directions therefore give equiconsistency. This proof does not replace the whole domain by a standard cut, a quotient, or a coded subset. □

The inherited numerical and logical bridge has a Lean formalization in the pinned repository. The new fusion theorem and the association of its Python checker with its stated rules are written proofs with executable validation. No new Lean or Rocq verification is claimed.

7 Complete certificate checkers

Both delivered acceptance programs are standalone Python modules using only the standard library. Each reads one JSON value from standard input. Acceptance means exit status zero and the line `ACCEPT`. The document has exactly the keys `goal` and `proof`. The goal is a closed native formula, and the checker infers every intermediate conclusion from the proof tree. It compares the inferred final formula with the public goal. There are no trusted proof-producing helpers.

7.1 The complete public syntax

The same disjoint-by-sort encoding is used by both programs:

Object	JSON representation
Zero	`0`
Free parameter level	a negative integer
Bound variable index	a positive integer
Successor St	`[0,t]`
Iterator $I(n, a; u.h)$	`[n,a,h]`
Equation $s = t$	`[s,t]`
Paired formula DB	`[B]`

An iterator adds one bound index in its last child only; D adds two in its matrix. At a matrix's outer level, index 1 is its induction/closure variable and index 2 its witness variable. Integers are checked with exact type tests, so JSON Booleans are not accepted as numerical terms or indices. Constructor arities and the successor's zero tag are checked as well.

A clause is internally stored as a pair `(antecedent,body)`. An empty list in the antecedent position denotes its absence; an actual formula cannot be an empty list. The initial clause is `([], [1,1])`, exactly $\mathcal{R}$.

The three proof forms are

```
["u", clause_index, term, antecedent_proof]
["i", matrix, witness, successor_branch]
["e", major_proof, auxiliary_branch, formula_branch]
```

Use ignores its antecedent-proof payload for an unconditional clause. The payload may be any JSON value in that case; this is an intentional part of the format, not an unvalidated logical premise. Intro infers the paired formula from its supplied matrix. Discharge obtains the matrix by checking its major proof; its two branch conclusions determine the evaluation. No intermediate branch formula, replacement term or template is separately supplied to Discharge.

7.2 Scope and capture invariants

The proof starts with one clause and no free parameters. If a checking context contains n clauses, its available free levels are $1 - n, \ldots, -1$. Every Intro or Discharge creates exactly one new numerical parameter $b = -n$ and adds exactly one clause. This is why clause count can supply the scope bound in the closed-theorem interface. It is not a claim that arbitrary mathematical clause contexts determine their free variables.

Every recursive proof result is checked for its formula sort and numerical scope. A discharge result is checked in the *outer* scope, after its elementary evaluation. This admits a valid inference in which evaluation erases the witness from the intermediate formula; it rejects every inference whose exported formula still contains the witness. Premises are independently checked in their common local scope before evaluation.

Every term substituted by the implementation is externally scoped and has no loose positive index. Its internal positive indices are already under its own binders. Consequently inserting it below an additional binder requires no index shift. The shared traversal only adjusts the index of the opened binder and decrements indices above it. In the successor Eval, the body comes from the

root iterator of an equation, so there is precisely one open iterator variable to instantiate and no hidden surrounding formula binder. The zero companion has the same invariant.

The checker does not validate an Intro matrix in a separate preliminary pass. It validates the complete original matrix in the synthesized result $[B]$ before returning successfully. Thus malformed material cannot become acceptable merely because branch substitution erases it. The adversarial suite includes such an attempt in the zero-normal variant.

7.3 Primitive-recursive checking without fuel

On standard finite-tree encodings, raw substitution is primitive recursive. For tree-node size, a conservative bound is

$$|\operatorname{sub}(a,t)| \le |a| \max(1,|t|).$$

The traversal visits the original tree and does not traverse newly inserted copies recursively. A raw successful evaluation performs either no substitution or one body substitution. The zero companion adds only structural deletions and at most one constructed predecessor iterator.

Every recursive proof call consumes a proper subcertificate. The finitely many traversals and substitutions at one node have a fixed polynomial majorant in the sizes of that node's input data, retained context and already inferred subproof conclusions. Iterating a generous such majorant over the finite certificate tree gives primitive-recursive bounds on all intermediate sizes and verification time. The ideal certificate predicate is therefore primitive recursive in both presentations, with no additional fuel field.

This differs from running a full normalizer on arbitrary primitive- recursive program descriptions. Neither checker asks for a uniform numerical evaluation of such descriptions or adds a totality assertion for one to iPRA. Ordinary Python parser, stack and memory limits still bound concrete runs; they are separate from this ideal finite-tree statement.

8 Measured comparison and executed evidence

All reported measurements include the entire acceptance module: parsing, syntax and scope validation, substitution, context operations, inference checking, final-goal comparison and the acceptance print. Under CPython 3.12.14, remove actual docstrings, apply `ast.unparse`, and add one final LF. Count UTF-8 bytes, significant tokens and Python AST nodes. Gzip uses level 9 and timestamp zero, and is informational. The bundled `measure.py` reproduces the table from the actual source files. Readable comments and documentation are not part of normalized source size.

Complete acceptance program	Bytes	Tokens	AST	Gzip9
Raw evaluation discharge	1892	643	695	773
Matched raw separate rules	1995	665	717	786
Zero-normal evaluation discharge	1858	633	697	777
Matched zero-normal separate rules	1879	645	707	771
Incoming inferred-witness output control	1983	663	751	812
Incoming inferred witness	1990	683	771	819
Incoming explicit clause discharge	2010	636	708	791

The isolated raw fusion saves 103 bytes, 22 tokens and 22 AST nodes (1,995 to 1,892 bytes, a 5.16% reduction). Its gzip size is also 13 bytes smaller. The isolated zero-normal fusion saves

21 bytes, 12 tokens and 10 AST nodes; its gzip size increases by six bytes. Against the incoming 1,983-byte output-control program, the complete zero companion saves 125 bytes (6.30%), 30 tokens and 54 AST nodes.

The matched controls use exactly the same syntax and public contract as their new checkers, with separate Eval/SeedEval and Open instructions. The difference from those controls isolates the rule fusion. Comparisons with the incoming full-normalization systems also change the term discipline and some implementation details; they must not be described as the isolated saving from fusion.

The zero companion improves all three primary coordinates of the three listed incoming strict clause-checker programs. The raw checker prioritizes having no normalization machinery: it has fewer bytes and AST nodes than those programs, but its token count exceeds the 636-token explicit clause-discharge program. Neither is asserted to dominate every possible representation or every checker in the repository. The separate compact codec in incoming `deterministic_opening` is 508 normalized bytes, and its validated public-goal companion is 780 bytes. They remain smaller by that coordinate and implement a different certificate interface.

The delivered raw suite translates all 33 pinned upstream raw certificates into both the fused and separate kernels. It checks 38 accepted examples, 38 exact-goal fused/separate round trips, and 38 reverse expansions in an independently implemented upstream raw checker. It rejects 34 adversarial inputs and passes 144 standalone ordinary/optimized-Python CLI checks.

The zero suite replays all 85 pinned SeedEval certificates in their original checker, translates all 85 into each new kernel, checks both directions of the generic transformation and 85 fused round trips, and checks six additional positives and 27 adversaries. Its 66 standalone CLI outcomes also pass. The package-level reproduction runs both suites from their packaged paths and separately replays the independent reviewer fixtures.

The tests include actual use of an induction hypothesis, nested D formulas, transport through nested iterators, active witness erasure before scope exit, ordinary elimination through a compound auxiliary, mismatched seed/predecessor rejection, malformed constructors, Boolean indices, unbound variables and non-object JSON envelopes. Both ordinary Python and `python -O` are used. No trusted acceptance condition uses `assert`.

The independent reviewer constructed further fixtures separately from the main producers. The raw suite has five positives and seven adversaries, each checked with and without optimization (24 expected outcomes). The zero suite has two positives and four adversaries (12 expected outcomes). All passed. These are finite implementation checks and independent replay, not proof-assistant verification of the universal mathematical claims. The general equivalence and both interpretation composites are proved in the text.

9 Limitations and further research questions

The result removes an independent inference family and, in the main presentation, all normalization from the trusted term specification. It does not reduce arithmetic strength, discover a new primitive-recursive function basis, or establish an absolute shortest proof checker. All the inherited mechanisms have been credited: the scalar iterator, paired binder, optional-antecedent clauses, selected introduction, Eval and SeedEval, and the whole-domain arithmetic bridge.

The two computation patterns and identity fallback in ρ are explicit. The fixed reflexivity clause is explicit too. Moving a rule into an unmeasured library or treating a normalizer as free would make

an unfair comparison; neither occurs here. The raw and zero checkers are independent standalone acceptance programs, and their proof producers and translators are untrusted supporting artifacts.

Several concrete questions remain:

1. **Formalize the fusion itself.** Add a deeply embedded three-family derivability relation to the pinned Lean development, and prove its equivalence with `VPrf` by the two local translations in Theorem 3.3. The raw route should be shorter because no normal-form conversion invariant is needed.

2. **Verify the inference-synthesizing checker.** Prove the correspondence between its negative free levels, positive bound indices, clause-count scope invariant and the embedded calculus. Its small source size is useful only when all those obligations remain visible.

3. **Reduce proof duplication.** The reverse Open translation may copy its major proof into the witness scope as a neutral auxiliary. A DAG certificate could share that proof, but the cost of reference validation and scope control must be measured.

4. **Replace the identity fallback economically.** Can the elimination simulation use a single always-active arithmetic pattern with no extra template and without enlarging either the checker or the arithmetic assumptions? A rule-count reduction alone would not answer this.

5. **Compare a fixed common certificate contract.** The compact codecs and the strict tree checkers sit on different tradeoffs. A useful next experiment would fix public syntax validation, certificate grammar, host semantics and measurement convention, then search for genuine dominance under that common contract.

10 Reproduction and attribution

The archive includes standalone editable LaTeX, its compiled PDF, both complete acceptance programs, matched controls, normalized sources, proof transformations, examples and adversaries, measured results, independent-review evidence and source provenance. Python reproduction uses only the standard library:

```
python3 verify_manifest.py
python3 reproduce.py
python3 measure.py
python3 raw/checker.py < raw/examples/raw_zero_computation.json
python3 build_pdf.py
```

The PDF build requires the standard TeX packages named in the source and runs with shell escape disabled. The main LaTeX source is flattened and does not require the auxiliary section files. Rebuilding may change PDF metadata; the manifest describes the delivered bytes.

The following appendices reproduce, with attribution, the five mathematical foundation files from the pinned repository's `Research/certified_zero/reference/`. They provide the inherited logical and arithmetic bridge in the delivered document. Their references to an earlier "new" interface describe that historical interface, not an additional novelty claim of this report. The present native primitives are only those in Section 2, or the explicitly stated zero companion. Original copies and their hashes are retained in `reference/foundations/`.

A Complete raw acceptance checker

The following is the entire measured 1,892-byte raw module. The public term syntax is unrestricted raw syntax; the two arithmetic cases are visible in the discharge branch.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in (2, 1 if form else 3) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in enumerate(a)]
    return a

def syntax(a, n, form=False):
    need(walk(a, lambda i, d: need(1 - n <= i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: t if i == d + 1 else i - (i > d + 1), depth)

def prove(p, g):
    n = len(g)
    b = -n
    match p:
        case ['u', j, t, q]:
            need(type(j) is int and 0 <= j < n)
            syntax(t, n)
            u, B = g[j]
            if u:
                need(prove(q, g) == u)
            A = sub(B, t)
        case ['e', q, r, v]:
            D, = prove(q, g)
            B = sub(D, b, 1)
            local = g + [(sub(B, 0), B)]
            E = prove(r, local)
            A = prove(v, local)
            match (E, A):
                case [[s, t], [[0, a, h], z]] if s == a:
                    A = [z, t]
                case [[s, t], [[[0, c], a, h], z]] if s == [c, a, h]:
                    A = [z, sub(h, t)]
        case ['i', B0, t, r]:
            syntax(t, n)
            B = sub(B0, t, 1)
            u = sub(B, b if len(B0) == 2 and t == 0 else 0)
            need(prove(r, g + [([], u)]) == sub(B, [0, b]))
            A = [B0]
        case _:
            need(False)
    syntax(A, n, True)
    return A

def check(data):
    need(set(data) == {'goal', 'proof'})
    syntax(data['goal'], 1, True)
    need(prove(data['proof'], [([], [1, 1])]) == data['goal'])
if __name__ == '__main__':
    check(json.load(open(0)))
    print('ACCEPT')
```

B Complete zero-normal acceptance checker

The following is the entire measured 1,858-byte companion module. Its traversal deletes zero-counter iterators, and its discharge branch uses the single SeedEval pattern.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in (2, 1 if form else 3) and (form or size == 3 or a[0] == 0)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1) for i, b in enumerate(a)]
    return a[1] if size == 3 and a[0] == 0 else a

def syntax(a, n, form=False):
    need(walk(a, lambda i, d: need(1 - n <= i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: t if i == d + 1 else i - (i > d + 1), depth)

def prove(p, g):
    n = len(g)
    b = -n
    match p:
        case ['u', j, t, q]:
            need(type(j) is int and 0 <= j < n)
            syntax(t, n)
            u, B = g[j]
            if u:
                need(prove(q, g) == u)
            A = sub(B, t)
        case ['e', q, r, v]:
            D, = prove(q, g)
            B = sub(D, b, 1)
            local = g + [(sub(B, 0), B)]
            E = prove(r, local)
            A = prove(v, local)
            match (E, A):
                case [[s, t], [[[0, c], a, h], z]] if s == a:
                    A = [z, sub(h, t if c == 0 else [c, t, h])]
        case ['i', B0, t, r]:
            syntax(t, n)
            B = sub(B0, t, 1)
            u = sub(B, b if len(B0) == 2 and t == 0 else 0)
            need(prove(r, g + [([], u)]) == sub(B, [0, b]))
            A = [B0]
        case _:
            need(False)
    syntax(A, n, True)
    return A

def check(data):
    need(set(data) == {'goal', 'proof'})
    syntax(data['goal'], 1, True)
    need(prove(data['proof'], [([], [1, 1])]) == data['goal'])
if __name__ == '__main__':
    check(json.load(open(0)))
    print('ACCEPT')
```

C Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

C.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

C.2 Exact equivalence and the unused-witness construction

Lemma C.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem C.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y) \rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition C.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

C.3 Induction and numerical cases separate cleanly

Lemma C.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma C.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the

successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

C.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t,u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A,E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction

uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

C.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem C.2 preserves every native sequent.

C.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

D Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section C recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

D.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem D.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

E Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

E.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by

induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

E.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

E.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

E.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

E.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

F From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix E, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

F.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE

contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

F.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$

$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$

$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

F.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

F.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

F.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

F.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism,

with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

G Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

G.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

G.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

G.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section C give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] V. Reshetnikov, ed., *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, commit `11814771b7490dfbea0ed5fcf0f5b23586334764`. Pinned repository snapshot. The README, filed packages and six incoming archives define the inspected comparison frontier. Exact source paths and hashes are in `SOURCE_PROVENANCE.json`.

[2] ProveIt, *Raw evaluation and uniform clauses*, `Lean/ConciseIPRA/NativeEval.lean`, `NativeClausal.lean`, and `NativeUniformClauses.lean`, in [1]. In particular `eprf_iff_nprf`, `cprf_iff_nprf`, and `vprf_iff_cprf`. Uniform-clause source.

[3] ProveIt, *Seed evaluation and zero-normal clauses*, `Lean/ConciseIPRA/NativeZeroSeed.lean` and `NativeZeroClausal.lean`, in [1]. SeedEval source; `nvprf_iff_pra` and `zvprf_iff_nprf`.

[4] *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, in [1]. Reproduced with attribution in this report.

[5] *Arithmetic strength and both ordinary interpretation composites*, `Research/certified_zero/reference/inherited_ordinary_bridge.tex`, in [1]. Reproduced with attribution.

[6] *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, in [1]. Source of the factor map, comparison and saturated-iteration construction of predecessor; reproduced with attribution.

[7] *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, in [1]. Internal reflection of actual equality and the compilation of quantifier-free formulas; reproduced with attribution.

[8] *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, in [1]. Reproduced with attribution.